Fast KVzip: Efficient and Accurate LLM Inference with Gated KV Eviction

Jang-Hyun Kim*
NAVER AI Lab

Dongyoon Han
NAVER AI Lab

Sangdoo Yun
NAVER AI Lab

Abstract

Efficient key-value (KV) cache management is crucial for the practical deployment of large language models (LLMs), yet existing compression techniques often incur a trade-off between performance degradation and computational overhead. We propose a novel gating-based KV cache eviction method for frozen-weight LLMs that achieves high compression ratios with negligible computational cost. Our approach introduces lightweight sink-attention gating modules to identify and retain critical KV pairs, and integrates seamlessly into both the prefill and decoding stages. The proposed gate training algorithm relies on forward passes of an LLM, avoiding expensive backpropagation, while achieving strong task generalization through a task-agnostic reconstruction objective. Extensive experiments across the Qwen2.5-1M, Qwen3, and Gemma3 families show that our method maintains near-lossless performance while evicting up to 70% of the KV cache. The results are consistent across a wide range of tasks, including long-context understanding, code comprehension, and mathematical reasoning, demonstrating the generality of our approach. Project page: https://janghyun1230.github.io/fastkvzip/

1 Introduction

Transformer-based LLMs have revolutionized the landscape of artificial intelligence, solving complex challenges ranging from long-context processing to advanced reasoning [23, 10]. A cornerstone of this success is the attention mechanism, which allows models to effectively contextualize features across sequences [47]. To mitigate the computational cost of re-calculating attention features at every step, inference systems adopt KV caching [12].

However, the effectiveness of KV caching is increasingly strained by the growing demand for long-context inference. As the memory required to store KV features scales linearly with sequence length, KV caches become a dominant bottleneck during inference [28]. While KV cache eviction approaches alleviate this burden, they often face a critical dilemma: Methods with negligible compression overhead result in significant performance degradation [55, 30], while methods that preserve accuracy tend to incur prohibitive compression overhead [26]. For instance, KVzip [26] relies on the reconstruction process that doubles computation during prefill (see Figure 1b), limiting its practicality for latency-sensitive deployment.

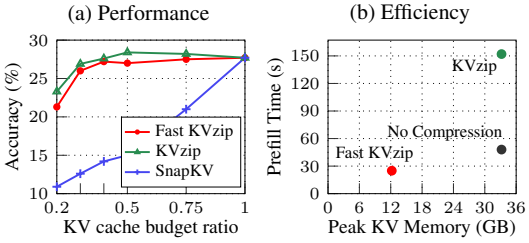


Figure 1: **Highlighted results.** (a) KV cache compression performance on OpenAI MRCR [37] compared to state-of-the-art methods using Qwen2.5-7b-1M [53]. (b) Chunked-prefill efficiency for 170K tokens on Qwen2.5-7b-14M (BF16 precision) with 30% KV cache budget.

*Currently at Apple. E-mail: kimjanghyun1230@gmail.com, sangdoo.yun@navercorp.com

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

In this paper, we introduce ***Fast KVzip***, a fast and accurate KV cache eviction method that addresses the inefficiencies of prior reconstruction-based approaches. Our key insight is that the future utilization of KV pairs is largely an *intrinsic property* that can be directly decoded from the input hidden states, without reconstructing the entire context as in KVzip. Leveraging this insight, we seamlessly integrate a lightweight gating mechanism into the Transformer's forward pass. This allows the model to directly assess the importance of KV pairs during both the prefill and decoding stages, incurring negligible memory and computational overhead while preserving high compression fidelity.

To identify an effective gating mechanism, we systematically explore the design space. Our empirical analysis reveals that gates perform most effectively when operating on input hidden states rather than intermediate attention features. Furthermore, we find that an architecture inspired by attention sinks [49] yields the strongest performance. Accordingly, we introduce a layer-wise gate training procedure that distills KV importance scores pre-computed from a compute-intensive reconstruction process. To ensure efficiency and avoid degrading the original model's capabilities, we keep the LLM weights frozen and train only the gating parameters, requiring less than one H100 GPU hour for 14B-scale models.

As highlighted in Figure 1, Fast KVzip matches the compression performance of KVzip while maintaining model accuracy at KV budget ratios as low as 40% on OpenAI MRCR [37]. Moreover, Figure 1b shows that Fast KVzip achieves approximately 5× speedup in 170K-token prefill over KVzip, while reducing prefill peak memory by roughly 3× compared to the no-compression baseline.

We further demonstrate the effectiveness and generality of our method across a diverse set of state-of-the-art open-source LLMs, including Qwen2.5-7B/14B-1M [53], Qwen3-8B/14B, Qwen3-8B-FP8 [52], and Gemma3-12B [46]. Our approach is compatible with quantized weights and sliding-window hybrid attention mechanisms. We evaluate on prefill-intensive long-context benchmarks such as SCBench [31] and MRCR [37] with context lengths of up to 170K tokens, as well as decoding-intensive reasoning tasks including AIME24 [2] and MATH [20]. Across all models and benchmarks, Fast KVzip achieves near-lossless performance while reducing the KV cache to 30–40% of its original size, underscoring the robustness and practicality of the proposed gating-based eviction strategy.

2 Related Work

Gating in Transformers. To address the scaling issues of Transformers [47], researchers have introduced gating mechanisms that enable sparse and selective computation, including MoE [44], MoD [41], and MoBA [36]. These approaches demonstrate that pretraining with gating can effectively exploit sparsity in MLP and attention modules, improving computational efficiency. Table 1 provides a conceptual comparison of these approaches with our method.

Table 1: **Gating strategies for Transformer attention.** The table below indicates operation execution per layer for a given input. Mixture-of-Depths (MoD) uses gating to skip attention computation, while Mixture-of-Block-Attention (MoBA) applies gating to select blocks for sparse attention with stored KV features. Our method bridges these approaches by conducting an attention operation for every input token while improving computational and memory efficiency through gated KV eviction.

Method	Attention	Store KV feature
MoD [41]	Conditional	Conditional
MoBA [36]	Always	Always
Fast KVzip	Always	Conditional

In the landscape of KV cache compression, Locret [22] employs a gating module to predict KV importance scores, while DMS [29] trains an eviction policy via logit distillation. TrimKV [5] introduces a retention-gated mechanism to learn time-decaying importance scores. Despite their promise, Locret is confined to prefill-stage compression, and TrimKV suffers from performance degradation in prefill-intensive scenarios, specializing in domains of mathematical reasoning. In our work, we provide a comprehensive interpretation of the gating mechanism, demonstrating that well-designed gating architectures and optimization strategies achieve near-lossless compression across both prefill and decoding stages. We provide a more detailed comparison to prior methods in Section C. We also note a concurrent work, KVzap [24], which explores a similar gating optimization objective but differs in specific architecture and training recipe. Together with our work, these parallel efforts underscore the promise of gating-based approaches for KV cache compression.

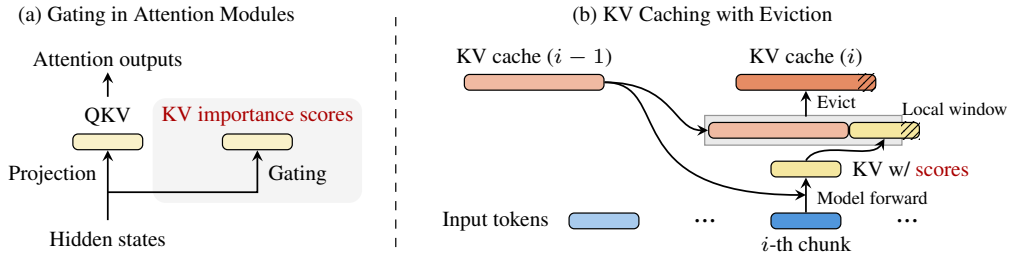


Figure 2: **Computational flow.** (a) Illustration of gating at each attention layer. (b) During each forward pass, we jointly calculate importance scores and evict low-importance KV pairs. For decoding, we maintain a small buffer to cache recent hidden states, performing gating and eviction in parallel once the buffer is full. This parallelized computation reduces latency overhead. We utilize a chunk size of 16K for prefill and a buffer size of 128 tokens for decoding. Empirically, we observe that retaining a local window of recent KV features at each compression step slightly improves performance (see Section B.5), though our method remains robust even without the local window.

KV Cache Compression. To mitigate the scalability bottleneck of KV caching in autoregressive Transformers [39], efficient attention mechanisms have been proposed, including sliding-window attention [9], linear attention [25], latent attention [32], and grouped-query attention [3]. While effective, these methods typically require training from scratch or substantial fine-tuning to accommodate architectural changes.

Recent work observes inherent sparsity in LLM attention patterns [34]. Methods such as H_2O [55], Finch [11], and SnapKV [30] exploit sparsity observed during prefill or decoding to evict underutilized KV pairs. However, sparsity patterns inferred at inference time often overfit the current input and fail to generalize across future queries. To address this limitation, query-agnostic approaches, *e.g.,* KVzip [26], EpiCache [27], and Expected Attention [15], estimate KV importance independently of specific queries and demonstrate improved robustness in multi-query settings. Orthogonal strategies, such as DuoAttention [50], RLKV [16], and FastGen [19], predefine attention patterns (e.g., sliding-window or A-shaped) using calibration datasets prior to deployment. Beyond token-level eviction, complementary compression techniques explore alternative dimensions, including feature-channel compression [51], low-rank compression [7], and KV quantization [35].

3 Method

3.1 Gating Mechanism

Formulation. We design a gating module that processes the hidden features of each attention layer and outputs importance scores for the KV features in that layer. Specifically, the gate at layer l is defined as $g_l : \mathbb{R}^D \to [0,1]^H$, where D denotes the hidden feature dimension and H is the number of KV heads in the layer. We introduce an additional gating branch into the model's forward computation. This branch operates independently and does not affect the original attention outputs (see Figure 2a).

Computational Flow. Figure 2b illustrates the overall computational flow of our method. We adopt a chunked prefill to reduce the peak memory usage when processing long contexts [1]. For each input chunk, we first compute the KV features and their corresponding importance scores using the gating mechanism. Based on these scores, we evict low-importance KV features from the cache, maintaining a compressed KV cache either at a fixed retention ratio or within a predefined memory budget. Our approach is compatible with both non-uniform and uniform KV cache structures [18, 30]. By maintaining a compressed KV cache throughout chunked prefill, our method reduces the peak KV cache memory compared to naive chunked prefill, as shown in Figure 1b.

During decoding, we apply the same gated KV eviction strategy. However, computing gates at every decoding step introduces substantial overhead due to frequent function calls and increased I/O, resulting in a 30–60% latency increase depending on the context length. To mitigate this issue, we maintain a small buffer of token length 128 that caches recent hidden states and periodically computes

gating in parallel. After updating the importance scores, we perform KV compression accordingly. This strategy reduces the gating overhead to approximately 1% of the model's forward-pass latency.

3.2 Interpretation of Gating Formulation

Before detailing our method, we establish a conceptual basis for why gating mechanisms are effective for KV cache compression. Notably, we observe that several prominent compression methods can be reformulated as constrained instances of a generalized gating framework (Figure 3).

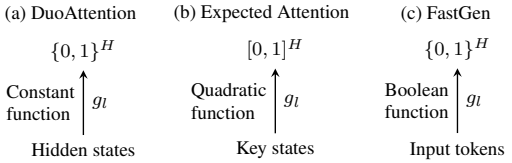


Figure 3: **Gating formulation** of existing KV compression methods at each attention layer. Note that we compare the core components of baseline methods, as each method jointly uses other techniques. For instance, FastGen combines its approach with cumulative attention scores [55].

For instance, DuoAttention [50] optimizes a head-level indicator function to designate specific heads for local-window attention. This represents a constrained gating formulation where the gating function is a constant Boolean mapping, independent of inputs. Expected Attention [15] approximates future attention scores by applying a quadratic model of key states with mean query-state statistics. Furthermore, FastGen [19] identifies heads that attend to specific token clusters (*e.g.*, punctuation tokens), which can be interpreted as a head-wise gating function on input tokens.

Our gating approach generalizes these prior methods by allowing any differentiable gating function without imposing empirical and heuristic constraints on the optimization. This interpretation enables a principled approach to KV cache compression, transforming the problem from heuristic design to data-driven optimization, effectively utilizing the model to discover the optimal compression strategy.

3.3 Gate Training

Consideration. We impose three requirements on our optimization: (1) computational efficiency, (2) robustness to task-specific overfitting, and (3) preservation of the base LLM's performance. End-to-end optimization via backpropagation, for example, incurs substantial memory overhead, limiting the usable context length and increasing training time. Furthermore, training gating mechanisms on data from a specific domain or task, such as mathematical reasoning [5] or instruction-based question answering [22], may yield domain-specific gains but incur degraded performance on other domains or tasks.

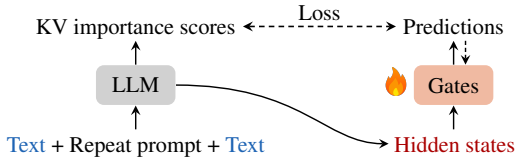
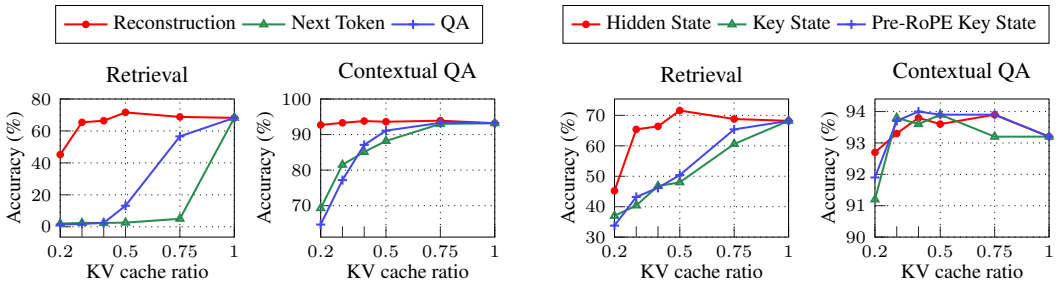


Figure 4: **Gate training process.** We compute the importance scores of KV pairs for a given text input following KVzip, which calculates the maximum attention score each KV pair receives during the context reconstruction [26]. We train gating modules to predict these scores from the corresponding input hidden states at each attention layer given the text. Please refer to Section A for further details and the training hyperparameters.

Training Procedure. With these considerations, we propose a gate training procedure illustrated in Figure 4. We optimize the gates at each layer using target scores derived from KVzip's reconstruction process [26], while freezing LLM weights to avoid altering the model's inherent capabilities. This design is computationally efficient, as reconstruction can be performed using parallelized forward passes. Given the target scores, we optimize gates independently at each layer using stochastic gradient descent with a binary cross-entropy loss [48]. The resulting optimization is embarrassingly parallel across both samples and layers, enabling efficient GPU execution.

Reconstruction-based target signals have been shown to generalize well to downstream tasks [14, 26], thereby mitigating task-overfitting issues. While KVzip introduces additional compression latency at inference time due to reconstruction, we perform this step only during a pre-deployment phase to train the gates. Consequently, inference remains fast.

Training Data. We train the gating mechanism using the FineWeb-Edu pretraining corpus [38], which has no overlap with downstream evaluation datasets. We randomly sample sequences ranging



(a) Effect of target scores (b) Effect of gate inputs

Figure 5: **Gate Training Design Analysis**. We evaluate gate training targets and inputs on Qwen2.5-7B-1M using a retrieval task (SCBench.KV) [31] and a contextual QA task (SQuAD) [40]. In (a), we compare different tasks to obtain target scores for gate training. For a fair comparison, we fix the gate architecture and only vary the task used to compute the maximum attention scores in our gate training (Figure 4). We use FineWeb-Edu for training data, except for QA where we use LongAlpaca [8] following Huang et al. [22].

from 10K to 30K tokens, yielding a total of 1M training tokens. Given that FineWeb-Edu contains over 1T tokens, our training set corresponds to approximately a 10^{-6} fraction of the full corpus. In Section B, we analyze the effect of data source and size to our method, demonstrating its robustness.

Alternatives. In Figure 5a, we compare the reconstruction-based target KV scores with scores derived from next-token prediction [5] and instruction-based question answering [22]. The figure demonstrates that reconstruction-based targets generalize significantly better, while alternatives lead to inferior performance.

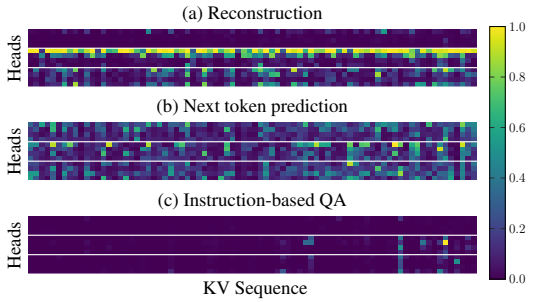


Figure 6: **Maximum attention scores** received by KV features across specific tasks. We visualize layers 7, 14, and 21 of Qwen2.5-7B-1M, with other layers exhibiting similar patterns, using the gate training data from Figure 5a.

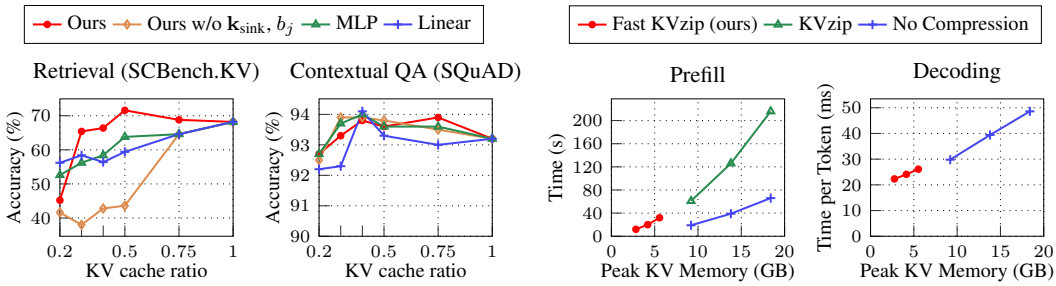
Figure 6 reveals that reconstruction-based attention patterns are uniform and structured across KV sequences. In contrast, next-token prediction generates denser patterns, reflecting more intensive feature contextualization. We hypothesize that the reconstruction task is effective because many downstream applications rely primarily on these high-level contextualized features [26]. To maintain necessary local dependencies, our method also preserves KV pairs within a sliding window (Figure 2). Conversely, QA tasks exhibit sparse attention as they focus only on query-specific information. Consequently, gates trained solely on QA signals generalize poorly, failing to retain the sufficient information required for broader downstream tasks.

In Figure 5b, we study the choice of gate inputs by comparing hidden states, key states, and pre-RoPE key states [45]. Hidden states outperform the alternatives, indicating that they contain richer information for predicting the future utility of KV features. We observe that incorporating position-encoded features degrades performance. This motivates decoupling positional information from the gating mechanism and instead leveraging recency through a simple local-window strategy (Figure 2).

3.4 Architecture

Consideration. The primary objective in designing the gate architecture is to minimize computational and memory overhead while accurately modeling attention behavior to predict the future utilization of each KV feature.

Low-Rank Sink Attention. We introduce a lightweight, attention-inspired gating mechanism, illustrated in Figure 8. Given a hidden state $\mathbf{h} \in \mathbb{R}^D$, we apply linear projections followed by



(a) Effect of gate architectures (b) Inference efficiency

Figure 7: (a) **Architecture analysis**. We compare our sink-attention, a variant without learnable denominator terms, a two-layer MLP, and a linear model, using Qwen2.5-7B-1M. For a fair comparison, we equalize the number of parameters across all architectures. (b) **Prefill and decoding efficiency** of the Qwen2.5-7B-1M model using a 30% KV budget ratio with PyTorch and FlashAttention-2 [13] on a single H100 GPU. Points in the plot correspond to context lengths of 160K, 240K, and 320K. In Section B.3, we provide an end-to-end latency analysis.

weighted normalization [52] to obtain $\mathbf{k} \in \mathbb{R}^{H \times D'}$ and $\mathbf{q} \in \mathbb{R}^{G \times H \times D'}$, where H is the number of original KV heads and G denotes the original grouped-query size [3]. Both representations are low-dimensional relative to the original KV features.

The gate produces an importance score $\mathbf{s} \in \mathbb{R}^{H}$ via a sink-attention mechanism as

$$\mathbf{s} = \frac{1}{G} \sum_{j=1}^{G} \frac{\exp(\mathbf{q}_j{}^{\intercal} \mathbf{k})}{\exp(\mathbf{q}_j{}^{\intercal} \mathbf{k}) + \underbrace{\sum_{r=1}^{S} \exp(\mathbf{q}_j{}^{\intercal} \mathbf{k}^{r}_{\text{sink}}) + b_j}_{\text{Learnable sinks}}}.$$

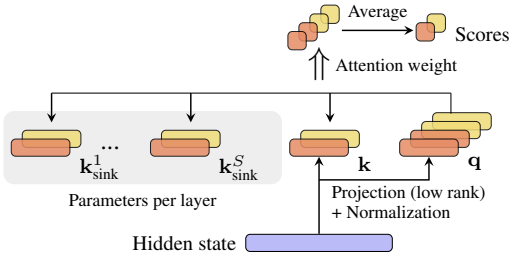


Figure 8: **Gate architecture.** We propose a sink-attention architecture using a fixed set of keys per layer. Given a hidden state $\mathbf{h} \in \mathbb{R}^{D}$, we compute the importance scores of KV as $\mathbf{s} \in [0, 1]^{H}$.

Here, $\mathbf{k}^r_{\text{sink}} \in \mathbb{R}^{H \times D'}$ are layer-specific learnable parameters that model sink keys, which play a critical role in standard attention mechanisms [49]. We empirically observe that these features enable the gate to effectively estimate the importance of KV features for future attention. A scalar $b_j \geq 0$ serves as a bias controlling head-level importance. These parameters are trained jointly with the projection and normalization weights.

Unlike DeepSeek-V3.2, which employs a low-rank KV cache within its Lightning Indexer to enable sparse attention [33], our method uses a fixed set of low-rank key features per layer and introduces no additional KV caching. This design simplifies implementation and enables straightforward integration into existing architectures.

Note that we use an identical set of hyperparameters across all models, setting $S = 16$ and $D' = 16$, which is $1/8$ of the Qwen3 head dimension [52], which suggests that our method requires neither exhaustive hyperparameter search nor manual fine-tuning. We provide a sensitivity analysis of hyperparameters in Section B.4.

Alternatives. We study alternative gate architectures, including a linear model, a two-layer MLP with SwiGLU activation [43], and a variant of our architecture without a learnable denominator, *i.e.,* setting $\sum_{r=1}^{S} \exp(\mathbf{q}_j{}^{\intercal} \mathbf{k}^r_{\text{sink}}) + b_j = 1$. Except for the linear model, we match the parameter counts across all architectures. Figure 7a shows that while alternative architectures are competitive on contextual QA tasks, they exhibit degraded performance on retrieval tasks. These results highlight the effectiveness of our sink-attention gate architecture.

3.5 Efficiency

Gate Training. Table 2 shows the training time and memory requirements of our gate training process, demonstrating its efficiency across diverse model scales.

Inference. Figure 7b shows the memory and latency efficiency improvements of Fast KVzip during both prefill and decoding. The results demonstrate that Fast KVzip reduces latency and memory usage compared to the base model, while significantly improving prefilling efficiency compared to KVzip by removing the runtime KV cache compression overhead. For all experiments, we apply a non-uniform KV cache eviction structure following Kim et al. [26].

Table 2: **Training efficiency.** Gate training time using a H100 GPU and gate sizes across diverse model scales. The 30B-A3B model has a smaller hidden state size than the other models, resulting in smaller gates.

Model	Training Time (h)	Storage (GB)
Qwen3-8B	0.59	0.18
Qwen3-30B-A3B	0.70	0.11
Qwen3-14B	0.83	0.30

4 Experiment

We evaluate Fast KVzip under both prefill-intensive and decoding-intensive scenarios. Note, we train gates once per model using FineWeb-Edu [38] and evaluate on all downstream tasks (see Section 3.3), demonstrating that the learned gates are task-generalizable. All gate training and model evaluations are conducted on a single NVIDIA H100 80GB GPU using PyTorch and FlashAttention-2 [13].

4.1 Setup

Datasets. For prefill-intensive scenarios, we evaluate all methods on SCBench [31], a multi-query benchmark comprising nine tasks, including synthetic key retrieval tasks from RULER [21], long-context question answering from LongBench [4], and code comprehension tasks. We further include OpenAI MRCR [37], a multi-round conversational retrieval benchmark, and SQuAD [40], a multi-query QA dataset. The maximum context length reaches up to 170K tokens (Qwen3 tokenizer).

For decoding-intensive tasks, we evaluate on mathematical reasoning benchmarks, AIME24 [2] and MATH [20]. Following the R-KV baseline [6], we set the maximum decoding length to 32K tokens for AIME24 and 16K tokens for MATH.

Models. We conduct experiments on state-of-the-art open-source LLMs, including Qwen2.5-7B/14B-1M [53], Qwen3-8B/14B, Qwen3-8B-FP8 [52], and Gemma3-12B [46]. We evaluate each model using its native precision. Notably, Gemma3-12B employs a hybrid sliding-window attention mechanism, while Qwen3-8B-FP8 adopts dynamic FP8 quantization. For the hybrid model, we only compress the global attention KV cache, which dominates memory usage in long-context evaluation

Baselines. We compare Fast KVzip against several state-of-the-art KV cache compression methods, including KVzip [26], SnapKV [30], Expected Attention [15] with Ada-KV [18], DuoAttention [50] with KVzip score [26], R-KV [6], and TrimKV [5]. For prefill-intensive tasks, we evaluate methods under the query-agnostic setting, following the setting of KVzip [26].

For decoding-intensive tasks, we adopt the R-KV setting [6], which continuously compresses the KV cache during decoding, covering both the context and generated tokens, to match the fixed-size cache budget. Additionally, we evaluate the early stopping of thinking strategy introduced in Qwen3 [52], where the model terminates the thinking process upon reaching a predefined budget and is forced to generate the final answer. Since some baselines are specifically designed for either prefill-intensive or decoding-intensive scenarios, we report comparisons only within their respective applicable settings.

4.2 Prefill-Intensive Tasks

Figure 9 shows the prefill KV cache compression performance across 12 datasets. The results demonstrate that Fast KVzip outperforms all baselines across all tasks while matching the performance of KVzip, which requires twice the prefilling computational cost for compression. The figure shows that Fast KVzip maintains full-cache performance at a 30–40% KV cache budget ratio. On some

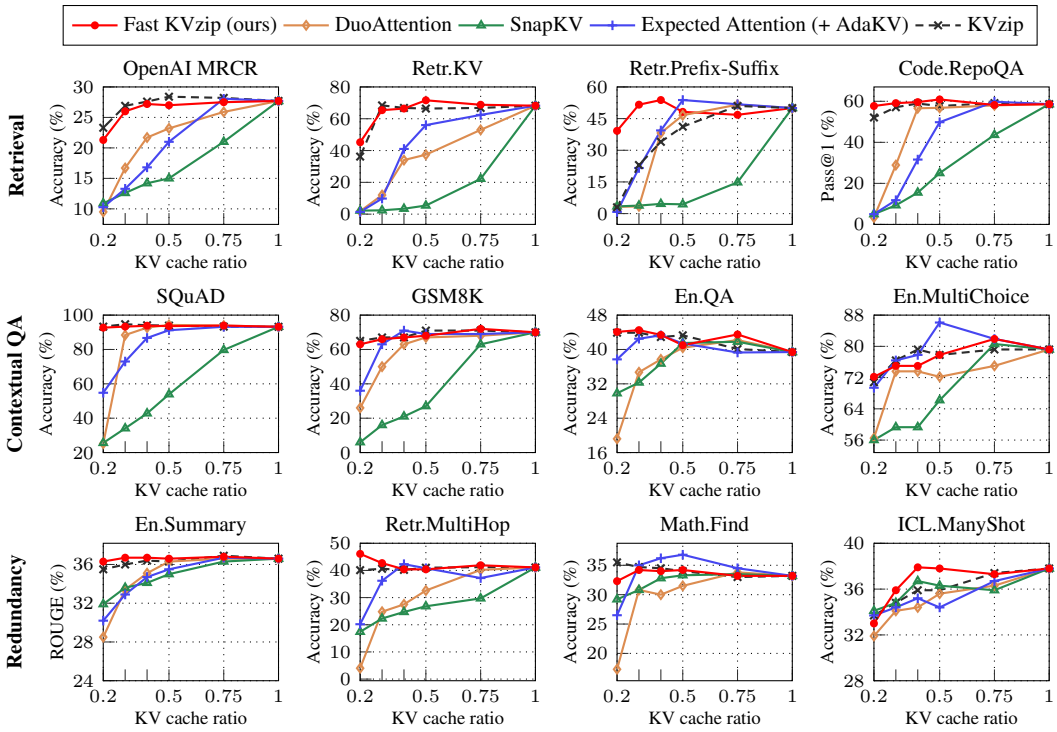


Figure 9: **Prefill-intensive benchmark results** with Qwen2.5-7B-1M under varying KV cache budget ratios from 0.2 to 1.0. Tasks are organized into three categories: (1) retrieval-intensive, (2) contextual understanding, and (3) high context redundancy.

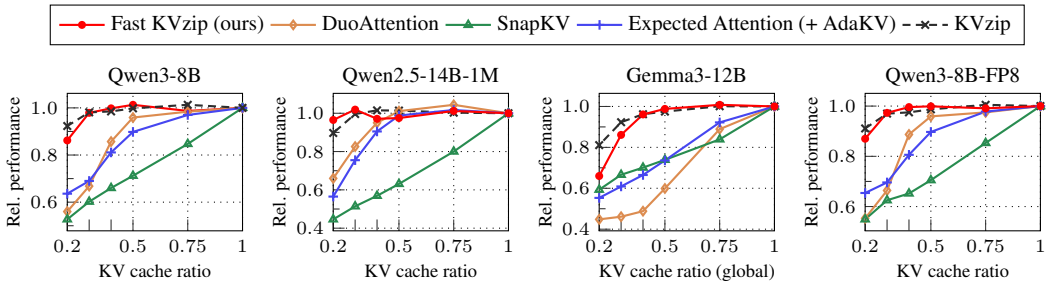


Figure 10: **Performance across different models**, averaged over 12 benchmark datasets. For each dataset, results are normalized with respect to full-cache performance prior to averaging.

tasks, Fast KVzip exhibits performance improvements, which we attribute to the denoising effect of attention induced by KV cache compression [54].

Figure 10 presents the averaged relative performance across the 12 datasets for various LLMs, demonstrating that Fast KVzip maintains effectiveness across models with a different pre-/post-training (Qwen3-8B), a larger model scale (Qwen2.5-14B-1M), a different attention mechanism (Gemma3-12B), and a quantization (Qwen3-8B-FP8).

4.3 Decoding-Intensive Tasks

Figure 11 shows the KV cache compression performance on mathematical reasoning tasks, including AIME24 and MATH. The results demonstrate that Fast KVzip achieves superior performance compared to decoding-scenario compression baselines on Qwen3-8B and Qwen3-14B models. Fast KVzip achieves near-lossless performance with a KV budget size of 4K, whereas the early-stopping-of-thinking method drastically reduces reasoning performance. These results indicate that

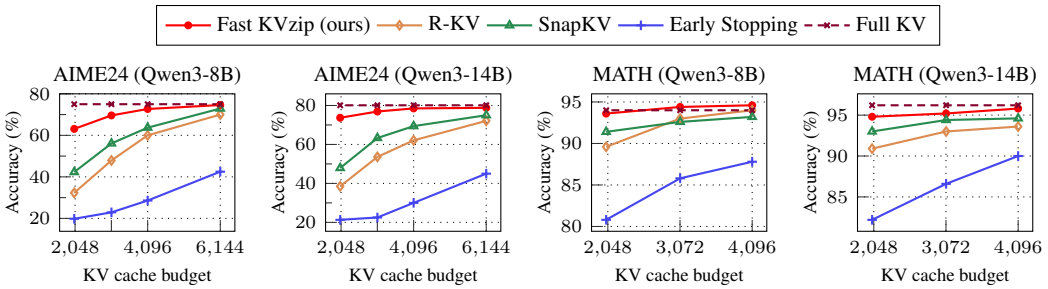


Figure 11: **Decoding-intensive tasks**. For AIME24, we averaged scores over 16 random seeds.

Table 3: **Gate behavior analysis.** The most frequently retained and evicted tokens for each head type. The \s represents a space.

Head Type	Most Retained (Top 1 → 8)	Most Evicted (Top 1 → 8)
Sparse	Structural markers (. .\n\n .\n , :\n \n \s)	High-frequency tokens (, the . and to of a I)
Medium	High-frequency tokens (, . and the of to I in)	High-frequency tokens (the , and to of a . I)
Dense	High-frequency tokens (, the . and to of a I)	Whitespace tokens (the \s , a \t \s\s (1)

the model requires a sufficient length of thinking to deduce correct answers, and that Fast KVzip indeed maintains the necessary KV features for general inference, including reasoning procedures.

5 Qualitative Analysis

We analyze the gating behavior of Qwen2.5-7B-1M using English book and code data from SCBench. Figure 12 illustrates the distribution of KV cache retention rates across attention heads under a total of 36% KV cache budget. We observe that half of the heads (primarily in early layers) exhibit highly sparse KV caches, while the remaining heads show uniformly distributed retention rates. This suggests that the gates prioritize high-level features from middle-to-late layers over low-level features from early layers.

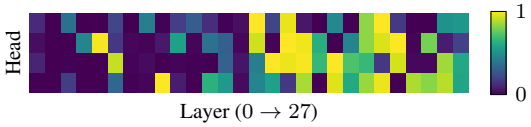


Figure 12: **Retention Rates per Attention Head.** We apply a 36% KV cache budget to Qwen2.5-7B-1M, which has 28 layers with 4 KV heads per layer. The left side corresponds to the early layers. Heads in the middle layers tend to retain the majority of token information, whereas gates in the early layers learn to retain only a small subset of KV features.

We categorize attention heads by their retention rates (r) into three types: sparse ($r < 0.05$), medium ($0.05 \leq r < 0.9$), and dense ($0.9 \leq r$). Table 3 details the tokens most frequently evicted and retained by each head type. Notably, sparse heads predominantly retain tokens associated with punctuation or line-break tokens. In contrast, other heads tend to retain tokens that are frequent in the source data; these frequent tokens appear as both the most retained and most evicted in medium heads. This complementary behavior across head types ensures that low-frequency but semantically important entities are preserved by the dense and medium heads, while sparse heads handle structural tokens, capturing high-level summarized information [42] which indicates an attention pattern distinct from naive local-window mechanisms [49].

6 Conclusion

We propose Fast KVzip, a gating-based KV cache compression method that matches state-of-the-art performance while eliminating runtime compression overhead. Through principled analysis, we design an effective gating optimization and a lightweight sink-attention architecture. Extensive evaluations on both prefill- and decoding-intensive tasks show that Fast KVzip generalizes well across diverse downstream applications, including retrieval, code comprehension, and reasoning, reducing KV cache size to 30% with negligible performance loss.

Acknowledgement

We thank the NAVER AI Lab team for valuable discussions. Jang-Hyun led the project and developed the main idea. Dongyoon and Sangdoo provided senior supervision and contributed to the methodology and experiments. Jang-Hyun and Sangdoo are corresponding authors.

References

[1] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee. Taming throughput-latency tradeoff in llm inference with sarathi-serve. *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.

[2] AIME. Aime problems and solutions, 2025. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions.

[3] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *EMNLP*, 2023.

[4] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, et al. Longbench: A bilingual, multitask benchmark for long context understanding. *ACL*, 2024.

[5] N. Bui, S. Sharma, S. Lamba, S. Mishra, and R. Ying. Cache what lasts: Token retention for memory-bounded kv cache in llms. *arXiv:2512.03324*, 2025.

[6] Z. Cai, W. Xiao, H. Sun, C. Luo, Y. Zhang, et al. R-kv: Redundancy-aware kv cache compression for training-free reasoning models acceleration. *Advances in Neural Information Processing Systems*, 2025.

[7] C.-C. Chang, W.-C. Lin, C.-Y. Lin, C.-Y. Chen, Y.-F. Hu, P.-S. Wang, et al. Palu: Compressing kv-cache with low-rank projection. *ICLR*, 2025.

[8] Y. Chen, S. Qian, H. Tang, X. Lai, Z. Liu, S. Han, and J. Jia. Longlora: Efficient fine-tuning of long-context large language models. *arXiv:2309.12307*, 2023.

[9] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv:1904.10509*, 2019.

[10] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv:2507.06261*, 2025.

[11] G. Corallo and P. Papotti. Finch: Prompt-guided key-value cache compression for large language models. *Transactions of the Association for Computational Linguistics*, 2024.

[12] Z. Dai, Z. Yang, Y. Yang, J. G. Carbonell, Q. Le, and R. Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019.

[13] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. *ICLR*, 2024.

[14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *NAACL*, 2019.

[15] A. Devoto, M. Jeblick, and S. Jégou. Expected attention: Kv cache compression by estimating attention from future queries distribution. *arXiv:2510.00636*, 2025. URL https://arxiv.org/abs/2510.00636.

[16] W. Du, L. Jiang, K. Tao, X. Liu, and H. Wang. Which heads matter for reasoning? rl-guided kv cache compression. *arXiv preprint arXiv:2510.08525*, 2025.

[17] H. Face. Open r1: A fully open reproduction of deepseek-r1, 2025. URL https://huggingface.co/datasets/open-r1/OpenR1-Math-220k.

[18] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference. *arXiv:2407.11550*, 2024.

[19] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *ICLR*, 2024.

[20] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv:2103.03874*, 2021.

[21] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. Ruler: What's the real context size of your long-context language models? *COLM*, 2024.

[22] Y. Huang, B. Yuan, X. Han, C. Xiao, and Z. Liu. Locret: Enhancing eviction in long-context llm inference with trained retaining heads on consumer-grade devices. *arXiv:2410.01805*, 2024.

[23] A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, et al. Openai o1 system card. *arXiv:2412.16720*, 2024.

[24] S. Jegou and M. Jeblick. Kvzap: Fast, adaptive, and faithful kv cache pruning. *arXiv:2601.07891*, 2026.

[25] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. *ICML*, 2020.

[26] J.-H. Kim, J. Kim, S. Kwon, J. W. Lee, S. Yun, and H. O. Song. Kvzip: Query-agnostic kv cache compression with context reconstruction. *Advances in Neural Information Processing Systems*, 2025.

[27] M. Kim, A. Kundu, H.-B. Kim, R. Dixit, and M. Cho. Epicache: Episodic kv cache management for long conversational question answering. *arXiv:2509.17396*, 2025.

[28] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[29] A. Łańcucki, K. Staniszewski, P. Nawrot, and E. M. Ponti. Inference-time hyper-scaling with kv cache compression. *Advances in Neural Information Processing Systems*, 2025.

[30] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 2024.

[31] Y. Li, H. Jiang, Q. Wu, X. Luo, S. Ahn, C. Zhang, A. H. Abdi, D. Li, J. Gao, Y. Yang, et al. Scbench: A kv cache-centric analysis of long-context methods. *ICLR*, 2025.

[32] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv:2405.04434*, 2024.

[33] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv:2512.02556*, 2025.

[34] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, et al. Deja vu: Contextual sparsity for efficient llms at inference time. *ICML*, 2023.

[35] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *ICML*, 2024.

[36] E. Lu, Z. Jiang, J. Liu, Y. Du, T. Jiang, C. Hong, et al. Moba: Mixture of block attention for long-context llms. *arXiv:2502.13189*, 2025.

[37] OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2025. URL https://huggingface.co/datasets/openai/mrcr.

[38] G. Penedo, H. Kydlíček, A. Lozhkov, M. Mitchell, C. A. Raffel, L. Von Werra, T. Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024.

[39] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.

[40] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. Squad: 100,000+ questions for machine comprehension of text. *EMNLP*, 2016.

[41] D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. C. Humphreys, and A. Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv:2404.02258*, 2024.

[42] A. Razzhigaev, M. Mikhalchuk, T. Rahmatullaev, E. Goncharova, P. Druzhinina, I. Oseledets, and A. Kuznetsov. Llm-microscope: Uncovering the hidden role of punctuation in context memory of transformers. *arXiv:2502.15007*, 2025.

[43] N. Shazeer. Glu variants improve transformer. *arXiv:2002.05202*, 2020.

[44] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv:1701.06538*, 2017.

[45] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.

[46] G. Team, A. Kamath, J. Ferret, S. Pathak, N. Vieillard, et al. Gemma 3 technical report. *arXiv:2503.19786*, 2025.

[47] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

[48] R. Wali. Xtreme margin: A tunable loss function for binary classification problems. *arXiv:2211.00176*, 2022.

[49] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. *ICLR*, 2024.

[50] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *ICLR*, 2025.

[51] Y. Xu, Z. Jie, H. Dong, L. Wang, X. Lu, A. Zhou, A. Saha, C. Xiong, and D. Sahoo. Think: Thinner key cache by query-driven pruning. *The Thirteenth International Conference on Learning Representations*, 2025.

[52] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, et al. Qwen3 technical report. *arXiv:2505.09388*, 2025.

[53] A. Yang, B. Yu, C. Li, D. Liu, F. Huang, H. Huang, et al. Qwen2.5-1m technical report. *arXiv:2501.15383*, 2025.

[54] T. Ye, L. Dong, Y. Xia, Y. Sun, Y. Zhu, G. Huang, and F. Wei. Differential transformer. *ICLR*, 2025.

[55] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 2023.

A Experimental Details

In this section, we detail the training procedure in Figure 4, describe the training hyperparameters used in Section 3.3, and the gate architecture detailed in Section 3.4.

A.1 Gate Training Target Score

In Figure 4, we construct the input as "<text> Repeat the previous context <text>" for each text sample from our training data, following KVzip [26]. Through the LLM forward pass, the repeated text attends to the keys of the original text. For simplicity, let us assume the number of attention heads as 1. From the attention matrix A^l at layer l, we compute importance score of each key states of the original text by the maximum attention it receives:

$$s_j^l = \max_{i \in \text{repeated}} A^l[i, j],$$

where i ranges over the repeated text positions and j indexes keys in the original text. Intuitively, a key that no repeated query attends to is safely evictable, while a key that at least one query strongly attends to should be retained.

We then train a lightweight gate g^l at each layer to predict these scores directly given the corresponding input hidden states h_j^l. Given target scores s_j^l, we optimize the gate using binary cross-entropy loss:

$$\mathcal{L}^l = -\sum_j \left[s_j^l \log g^l(h_j^l) + (1 - s_j^l) \log(1 - g^l(h_j^l)) \right],$$

while freezing all LLM weights. The optimization is embarrassingly parallel across both samples and layers, enabling efficient GPU execution.

A.2 Dataset

As described in Section 3.3, we use the FineWeb-Edu dataset [38] to train the gates. We randomly subsample sequences with context lengths ranging from 10K to 30K tokens, for a total of 500K tokens. By concatenating these samples, we construct an additional long-context dataset, yielding sequences of length 100K tokens and totaling another 500K tokens. In total, we use 1M tokens for gate training.

A.3 Training Hyperparameters

For efficiency, we first precompute tuples of hidden states and their corresponding KV importance scores using the dataset described above. We then optimize the gates in parallel using stochastic gradient descent with a binary cross-entropy loss and a learning rate of 0.2 across all experiments. Training is performed for 5K update steps with a batch size of 1K. Given the total of 1M training tokens, this corresponds to 5 epochs over the dataset.

Figure 13 illustrates the training loss trajectory and demonstrates stable convergence. We observe that earlier layers tend to exhibit lower loss, whereas the middle and later layers show higher loss values. We conjecture that higher layers perform more complex attention mechanisms, which are inherently more difficult to predict from a single hidden state.

Figure 18 presents a token-level comparison of predictions, illustrating that our trained gating mechanism captures token-level dynamics and avoids over-smoothing.

A.4 Gate Configuration

As described in Section 3.4, each gate uses 16 $\mathbf{k}_{\text{sink}}^s$ vectors. The low-rank projection dimension D' is set to 16 for all experiments.

A.5 Inference Details

During prefill, we employ chunked prefill with a chunk size of 16K tokens. As illustrated in Figure 2, KV cache eviction is performed after each prefill chunk, while retaining KV features from the most

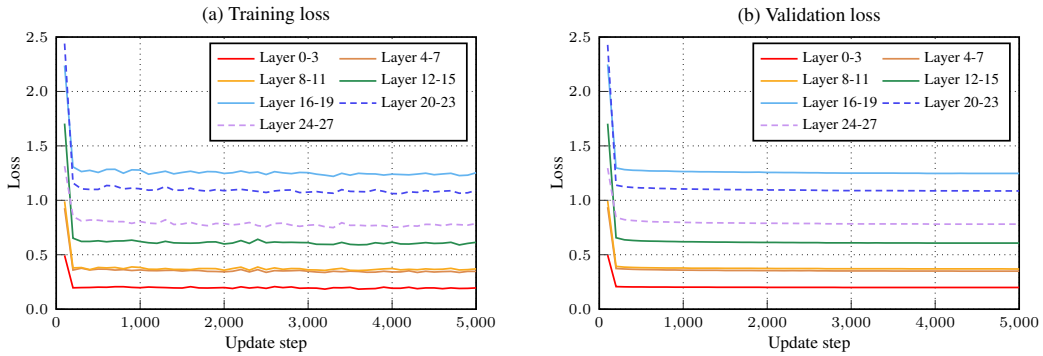


Figure 13: Gate training and validation loss trajectory per attention layer for Qwen2.5-7B-1M. We plot loss values averaged every four layers in the model.

recent 4K-token window to exploit recency bias in attention. For contexts shorter than 16K tokens, we retain the last 2% of tokens to accommodate arbitrary input lengths.

During decoding, KV eviction begins once the cache reaches the predefined budget. We use a hidden cache of size 128 and compute gating decisions in parallel. After updating importance scores, we evict an equivalent number of KV entries to maintain the cache budget. A local attention window of 128 tokens is preserved at each eviction step during decoding.

B Additional Experiments

B.1 Training Data Source

To analyze how training data affects gate behavior, we train a gate on OpenR1-Math data [17] with the identical training configuration, and evaluated on AIME24 [2]. As shown in Table 4, the math-domain gate performs comparably to our general-corpus gate. This result is intuitive: since our reconstruction-based objective trains the gate to preserve the context itself rather than to learn task-specific features, the choice of training domain has minimal impact, rather the broad coverage of FineWeb-Edu yields better generalization. This explains why our approach generalizes well across diverse tasks, while next-token prediction-based methods like TrimKV [5] exhibit strong performance specifically on the domain of their training data.

Table 4: Effect of gate training data on AIME24 (Qwen3-8B).

KV Cache Budget	Fast KVzip (OpenR1 Math)	Fast KVzip (FineWeb-Edu)	SnapKV	R-KV
4096	70.6%	72.7%	63.7%	60.0%
2048	53.3%	63.1%	42.3%	32.3%
512	16.7%	13.3%	0.8%	0.0%

B.2 Training Data Size

Figure 14 shows the performance of gates trained with different sizes of FineWeb-Edu data, as described in Section A.2. The figure demonstrates that increasing the size of the training data leads to performance improvements, particularly on SCBench.KV, the synthetic key-retrieval task. To further evaluate the effect of long-context data, we construct the 500K-short dataset by removing the long-context samples (each with a context length of 100K) from the 1M-token dataset described in Section A.2. The results demonstrate the effectiveness of incorporating long-context data for gate training.

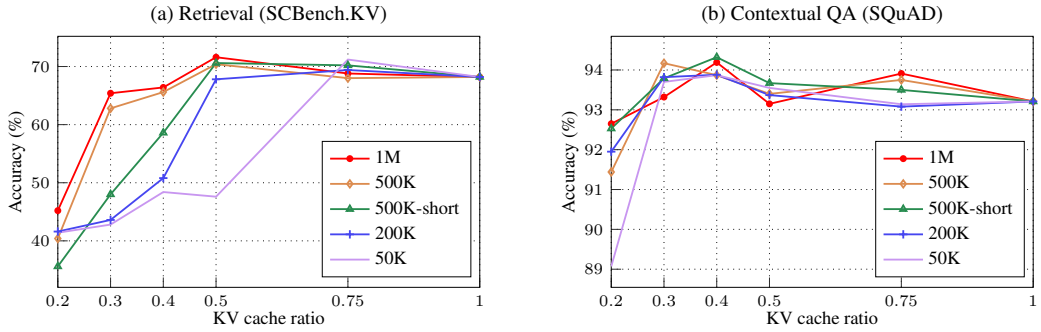


Figure 14: **Effect of Training Data Size.** We evaluate gates trained with different amounts of training data sampled from FineWeb-Edu using Qwen2.5-7B-1M. For the other settings, we uniformly reduce the context lengths of the data.

B.3 End-to-End Latency

We provide end-to-end results on SCBench-KV, the longest-context benchmark in our evaluation ($\sim$170K tokens). As shown in Table 5, KVzip incurs nearly 3$\times$ end-to-end latency compared to vanilla due to its reconstruction overhead. FastKVzip eliminates this overhead entirely, achieving up to 5$\times$ speedup over KVzip at 30% KV ratio while maintaining comparable accuracy. This speedup stems from two sources: (1) gating removes the per-query reconstruction cost, and (2) chunked prefill compression (Figure 2) further accelerates the prefill stage itself.

Table 5: End-to-end latency and accuracy on SCBench-KV with Qwen2.5-7B-1M. Acceleration ratios in parentheses are relative to vanilla (ratio 1.0). FastKVzip eliminates reconstruction overhead entirely, achieving up to 5$\times$ speedup over KVzip while maintaining comparable accuracy.

KV Ratio	FastKVzip Time (s)	KVzip Time (s)	FastKVzip Acc	KVzip Acc
1.00	22.5 (1.0$\times$)	69.1	68.2	68.2
0.50	16.8 (1.3$\times$)	68.7	71.6	68.6
0.40	15.0 (1.5$\times$)	68.4	66.4	66.0
0.30	13.1 (1.7$\times$)	68.2	65.4	65.0

B.4 Gate Configuration

Figure 15 analyzes the effects of the number of sink keys S and the projection dimension D' in our sink-attention architecture (Section 3.4). The figure shows that performance is robust to the choice of projection dimension, while highlighting the importance of having a sufficient number of sink keys for effective KV cache compression.

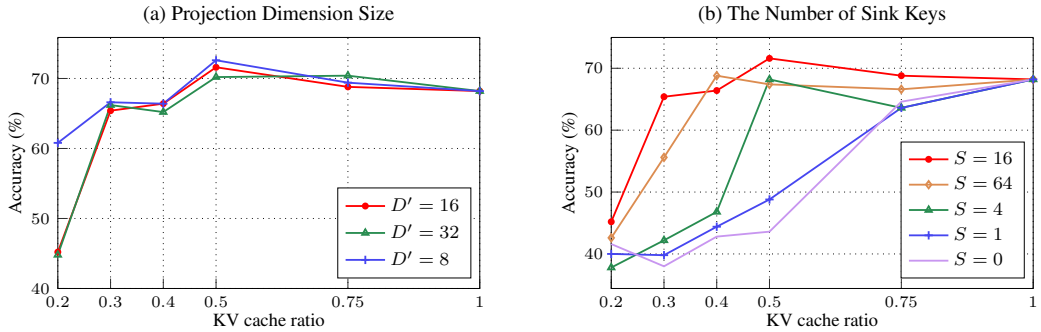


Figure 15: **Effect of Gate Configuration.** We evaluate gates with different numbers of sink keys S and projection dimensions D' using Qwen2.5-7B-1M on SCBench.KV.

B.5 Inference Configuration

Figure 16 shows the effect of the local window size in our inference algorithm described in Figure 2. The figure demonstrates that retaining a local window improves performance, while performance is comparable across local window sizes ranging from 0K (no local window) to 8K tokens.

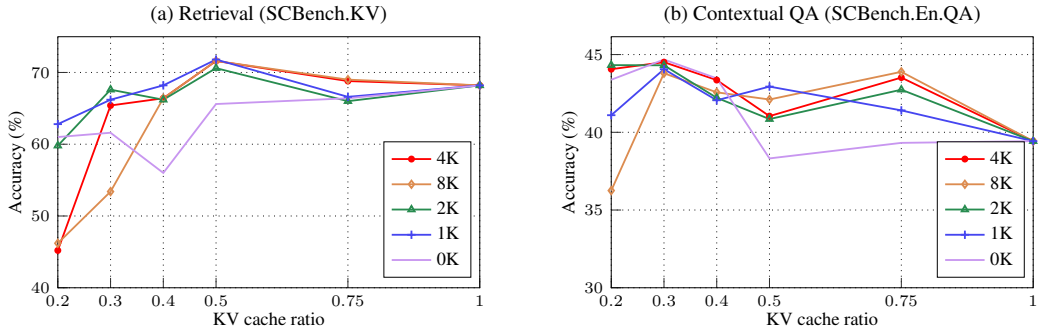


Figure 16: **Effect of Local Window Size.** We evaluate gates with a range of local window sizes, as described in Figure 2, using Qwen2.5-7B-1M. Note that our algorithm performs eviction that all configurations have an identical total KV cache size after eviction. In our main experiment, set the local window hyperparameter as 4K tokens for prefill and 128 tokens for decoding.

C Extended Comparison with Prior Gating Approaches

In Table 6, we provide a comparison with prior gating-based approaches for KV cache compression, including Locret [22], DMS [29], and TrimKV [5].

In Figure 17, we further evaluate our method on Qwen3-4B-Instruct-2507, for which an official TrimKV model for general language tasks is available. For other released models, TrimKV trains specialized gating models for mathematical reasoning using math-specific datasets, which prevents a fair comparison with our method, trained on general data in a task-agnostic manner. As shown in Figure 17, our Fast KVzip outperforms TrimKV in general task settings, particularly on retrieval tasks. These results demonstrate the effectiveness of our gating optimization and design, emphasizing their importance.

Table 6: **Comparison of gating approaches** for KV cache compression. While DMS and TrimKV trains separate gates for reasoning and instruction-following tasks using distinct datasets, we employ a single dataset for gate training across all models and evaluations.

Method	Training Data	Task	Target	Gate Architecture
Locret [22]	Instruction-based QA	Question-answering	Attention scores	MLP
DMS [29]	Mathematics (or Programming)	Next-token logit distillation	End-to-end loss	Linear
TrimKV [5]	Mathematics (or Book)	Next-token prediction	End-to-end loss	MLP
Fast KVzip (Ours)	General text data	Reconstruction	Attention scores	Sink attention

D Broader Impacts and Limitations

Broader impacts. This paper explores technical methods to enhance LLM efficiency and does not present direct social risks. Socially, our findings have a positive impact by decreasing the costs associated with deploying and maintaining large-scale models. By minimizing resource consumption, this work facilitates more sustainable and cost-effective AI operations for the broader community.

Limitations and Future Work. Prior gating methods for attention modules, such as MoD [41] and MoBA [36], typically require from-scratch training to fit predefined structures. An important future direction is to study how applying our method during pretraining can induce hardware-efficient structured computation patterns. Moreover, exploring a unified framework that integrates these gating

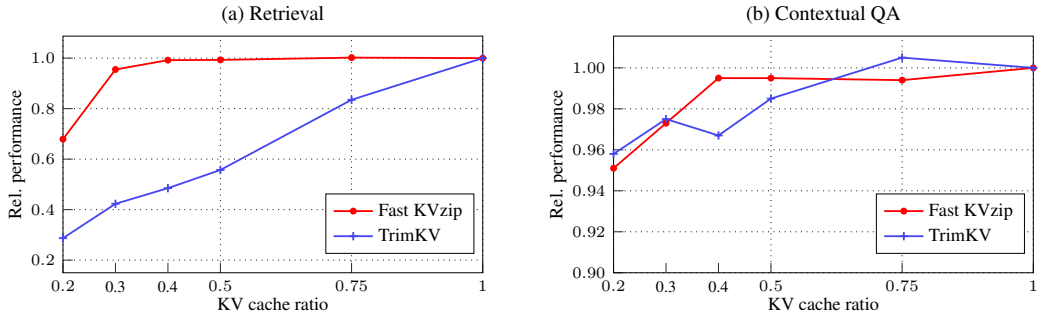


Figure 17: **Comparison to TrimKV** [5] with prefill-intensive tasks. We evaluate the official TrimKV model with Qwen3-4B-Instruct-2507 on retrieval and contextual QA tasks in Figure 9, and report the averaged scores. For each dataset, results are normalized with respect to full-cache performance prior to averaging. For Locret, the released models are outdated, and we replace performance comparisons with our analysis in Figure 5.

approaches is a promising avenue. In particular, the gating mechanism could be extended to support multi-choice decisions, including skipping computation, discarding KV features, or storing them for selective retrieval.

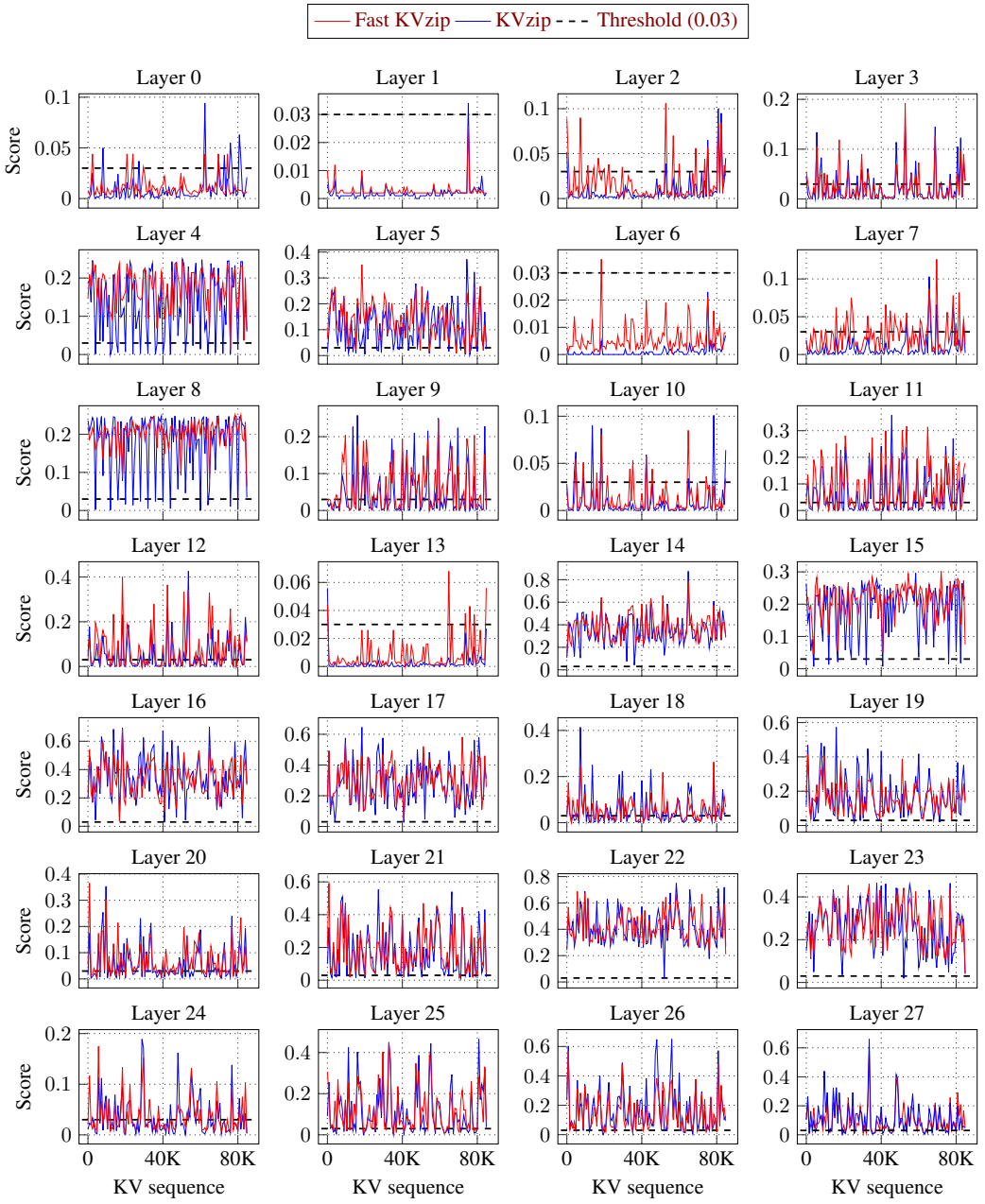


Figure 18: **Prediction Comparison of Fast KVzip to KVzip.** We use Qwen2.5-7B-1M with the SCBench.En.QA dataset, which were not used during training. For each layer, we plot the averaged score over heads. The dashed line represents the threshold value that results in a 36% KV cache budget ratio.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: In the abstract and the introduction (especially, the third paragraph), we describe our scope as KV cache compression and state our technical contributions.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: In Appendix D.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.

- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: We do not provide theoretical results.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 3 and Appendix A.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.

- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes] Data / [No] Code

   Justification: We use open-sourced training data (Appendix A) and evaluation data (Section 4). We plan to release our code in the camera-ready version.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We follow the convention. We run multiple seeds and report averaged score for math reasoning (AIME24, Section 4 Figure 11).

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Section 3.5 and Section 4.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?

Answer: [Yes]

Justification: Yes, we follow the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Appendix D.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: N/A

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [N/A]

    Justification: We use open-sourced models and datasets valid for the research purposes.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: N/A

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: N/A

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: N/A

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: We used the LLM for editing of our writing.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.